

OFFENSIVE SECURITY

Manual Reconnaissance Assessment Report

Target: OWASP Juice Shop (Intentionally Vulnerable Web Application)

Target URL: <http://localhost:3000>

Environment: Local Lab (Docker)

Engagement Phase: Reconnaissance Only

Prepared by: Dharmik

Date: August 5, 2026

Classification: Internal / Confidential

1. Executive Summary

This report documents a manual reconnaissance assessment performed against OWASP Juice Shop, an intentionally vulnerable web application deployed in a local, isolated lab environment via Docker and accessed at <http://localhost:3000>. The engagement was scoped strictly to the Reconnaissance phase of the penetration testing lifecycle.

The objective was to build a complete, evidence-backed picture of the target's exposed attack surface using passive and lightly active reconnaissance techniques, while documenting the workflow, tooling, and decision-making process in enough detail that the activity could later be automated. Eleven discrete reconnaissance activities were carried out, spanning environment setup and verification, HTTP header and technology fingerprinting, robots.txt/sitemap.xml discovery, browser-based traffic and client-side source analysis, cookie/storage inspection, and automated crawling.

Key outcomes include identification of the underlying technology stack (Node.js/Express backend serving an Angular single-page application), enumeration of client-side API endpoints and static assets, discovery of a disallowed path in robots.txt, confirmation that sitemap.xml is not a true XML sitemap but is intercepted by the SPA router, and observation of session/state data held in browser cookies and storage. No exploitation, data modification, or intrusive testing was performed at any point; all findings were gathered through observation of normal application behavior and industry-standard read-only reconnaissance tooling.

2. Complete Reconnaissance Workflow

Each activity below follows a consistent structure: Objective, rationale, input, tool, exact command executed, observed output, analysis, and the decision that drove the next step. Screenshots captured during testing are included as evidence.

Activity 1: Environment Setup – Deploying the Target Application

Objective: Stand up a local, isolated instance of the target application to test against.

Why: A real engagement always confirms the target environment is correctly provisioned before any testing begins; using a containerized, intentionally vulnerable app keeps testing legal and safe.

Input: Docker image name: bkimminich/juice-shop

Tool: Docker

Command: `docker pull bkimminich/juice-shop && docker images`

Output: Image pulled successfully (all layers 'Pull complete'); docker images confirms bkimminich/juice-shop:latest present, 338MB.

Observation: Image downloaded cleanly with no errors; a second unrelated image (hello-world) is also present locally.

Why it matters: Confirms the exact, unmodified vulnerable application is available locally before any recon is run against it.

Decision: Proceed to run the container and verify the application is reachable on port 3000.

```
(dharmik@kali)-[~]
$ docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
47de5dd0b812: Pull complete
c172f21841df: Pull complete
99515e7b4d35: Pull complete
99ba982a9142: Pull complete
d6b1b89eccac: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
52630fc75a18: Pull complete
dd64bf2dd177: Pull complete
b839dfae01f6: Pull complete
ebddc55facdc: Pull complete
bdfd7f7e5bf6: Pull complete
2d4d7adf6272: Pull complete
cf397222899a: Pull complete
bd8962e29291: Pull complete
cac2ae0193cb: Pull complete
dd0d9bfd3bee: Pull complete
7cc70dfd88cf: Pull complete
8224d91da70b: Pull complete
1a4be5562d92: Pull complete
702a7246bd56: Pull complete
48fce22aed6d: Pull complete
6b2d9fb43c80: Pull complete
Digest: sha256:e68144772ebaaca0ec117b38d44903af92416793230288ef7c5437fc4f26850a
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest

(dharmik@kali)-[~]
$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
bkimminich/juice-shop latest      9f0f27a44514  5 weeks ago    338MB
hello-world         latest     e2ac70e7319a  4 months ago   10.1kB
```

Figure 1 – Docker image pull and local image listing.

Activity 2: Reachability & Visual Verification

Objective: Confirm the target application is running and reachable, and record a baseline view of its UI.

Why: Before running any tooling, a tester must visually confirm the app loads correctly — this also gives a baseline understanding of the application's functionality (storefront, products, basket, account).

Input: <http://localhost:3000>

Tool: Web browser (Firefox, Kali Linux)

Command: N/A – manual navigation to <http://localhost:3000/#/>

Output: OWASP Juice Shop storefront loads, displaying a product catalog (Apple Juice, Apple Pomace, Banana Juice, Basil Smoothie, Berry Juice, etc.) with an Account, Basket, and language selector in the header.

Observation: The app is an Angular single-page application (SPA) — the URL uses a '#' hash-based route.

Why it matters: Establishes a functional baseline and confirms the app type (SPA), which shapes the recon approach (client-side JS review, DevTools-based traffic analysis).

Decision: Move to protocol-level inspection (HTTP headers) before exploring the UI further.

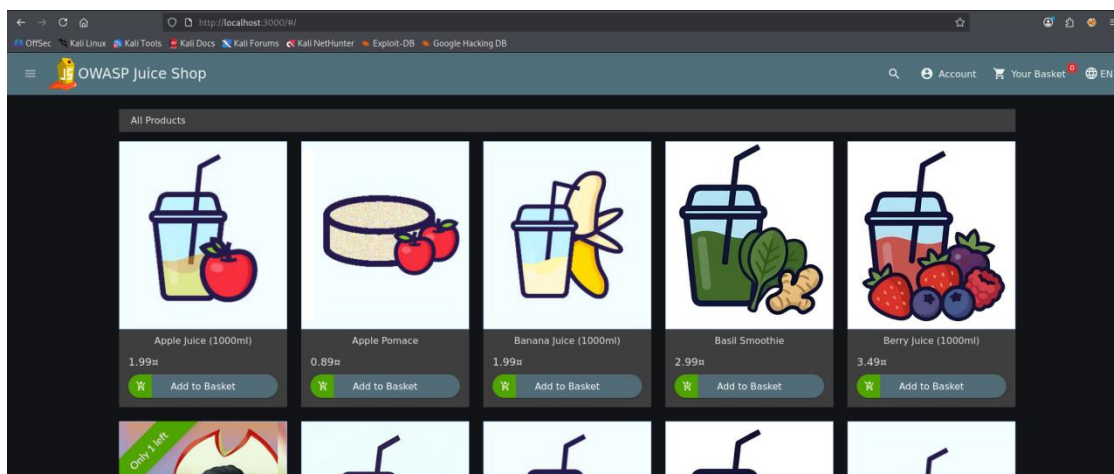


Figure 2 – OWASP Juice Shop storefront, confirming the target is live.

Activity 3: HTTP Header Inspection

Objective: Identify server technology, security headers, and basic response behaviour without rendering the page.

Why: Headers are the lightest possible recon check — they often reveal server software, framework hints, and misconfigurations.

Input: `http://localhost:3000`

Tool: `curl`

Command: `curl -I http://localhost:3000`

Output: HTTP/1.1 200 OK; Access-Control-Allow-Origin: *; X-Content-Type-Options: nosniff; X-Frame-Options: SAMEORIGIN; Feature-Policy: payment 'self'; X-Recruiting: /#/jobs; Accept-Ranges: bytes; Cache-Control: public, max-age=0; ETag, Content-Type: text/html; charset=UTF-8.

Observation: The application is reachable and returns a healthy 200 status. A non-standard 'X-Recruiting' header pointing to /#/jobs was noted — an intentional Juice Shop Easter egg, not a standard production header.

Why it matters: Confirms baseline availability and surfaces security-relevant headers (CORS wide open, clickjacking protection present) that describe the app's security posture without touching functionality.

Decision: Fingerprint the underlying technology stack in more depth using a dedicated tool.

```
(dharmik@kali)-[~]
$ curl -I http://localhost:3000
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Tue, 04 Aug 2026 13:14:38 GMT
ETag: W/"26af-19fcce96ec5"
Content-Type: text/html; charset=UTF-8
Content-Length: 9903
Vary: Accept-Encoding
Date: Tue, 04 Aug 2026 13:38:45 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Figure 3 – `curl -I` response headers for `http://localhost:3000`.

Activity 4: Technology Fingerprinting

Objective: Identify the frameworks, libraries, and server software powering the application.

Why: Knowing the exact stack narrows down what recon paths and later-phase testing are relevant, without needing to guess.

Input: `http://localhost:3000`

Tool: WhatWeb

Command: `whatweb http://localhost:3000`

Output: `http://localhost:3000 [200 OK] HTML5, IP[::1], Script[module], Title[OWASP Juice Shop], UncommonHeaders[access-control-allow-origin, x-content-type-options, feature-policy, x-recruiting], X-Frame-Options[SAMEORIGIN]`

Observation: Confirms HTML5/SPA structure and the same uncommon headers seen via curl; no explicit server banner (e.g. Express version) was disclosed.

Why it matters: Cross-validates the curl findings with a purpose-built fingerprinting tool, increasing confidence in the technology assessment.

Decision: Check for standard metadata/disclosure files (robots.txt, sitemap.xml) next.



```
(sharik@kali):~$ whatweb http://localhost:3000
http://localhost:3000 [200 OK] HTML5, IP[::1], Script[module], Title[OWASP Juice Shop], UncommonHeaders[access-control-allow-origin, x-content-type-options, feature-policy, x-recruiting], X-Frame-Options[SAMEORIGIN]
```

Figure 4 – WhatWeb fingerprinting output.

Activity 5: robots.txt Discovery

Objective: Check for a robots.txt file that may reveal paths the site owner does not want indexed.

Why: robots.txt is a purely passive, publicly-intended file — checking it carries no risk and can reveal internal structure.

Input: `http://localhost:3000/robots.txt`

Tool: Browser

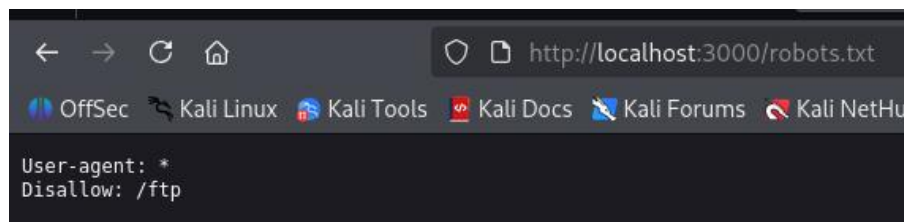
Command: Navigate to `http://localhost:3000/robots.txt`

Output: `User-agent: * Disallow: /ftp`

Observation: A single disallowed path, /ftp, is declared — an unusual entry for a modern web app and worth noting as a point of interest.

Why it matters: Disallowed paths sometimes point to legacy or sensitive functionality that isn't linked from the UI.

Decision: Check sitemap.xml for further structural disclosure.



```
← → ↻ 🏠 http://localhost:3000/robots.txt
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHu
User-agent: *
Disallow: /ftp
```

Figure 5 – robots.txt contents.

Activity 6: sitemap.xml Check

Objective: Check for an XML sitemap that might enumerate additional application routes.

Why: Sitemaps are a standard, passive source of structural information on traditional sites.

Input: `http://localhost:3000/sitemap.xml`

Tool: Browser

Command: Navigate to `http://localhost:3000/sitemap.xml`

Output: The request resolves to the normal Juice Shop storefront UI rather than XML content; the URL bar shows `sitemap.xml#/`, indicating the Angular router intercepted the path.

Observation: No genuine `sitemap.xml` exists — the SPA's client-side routing catches all unmatched paths and renders the default view instead of a 404 or a real sitemap.

Why it matters: Important negative finding: confirms the app relies on client-side routing rather than server-side static files for this path, which affects how automation should interpret 'success' responses (a 200/rendered page is not proof a resource exists).

Decision: Move from server-side file checks to client-side (browser DevTools) analysis to map real routes and API calls.

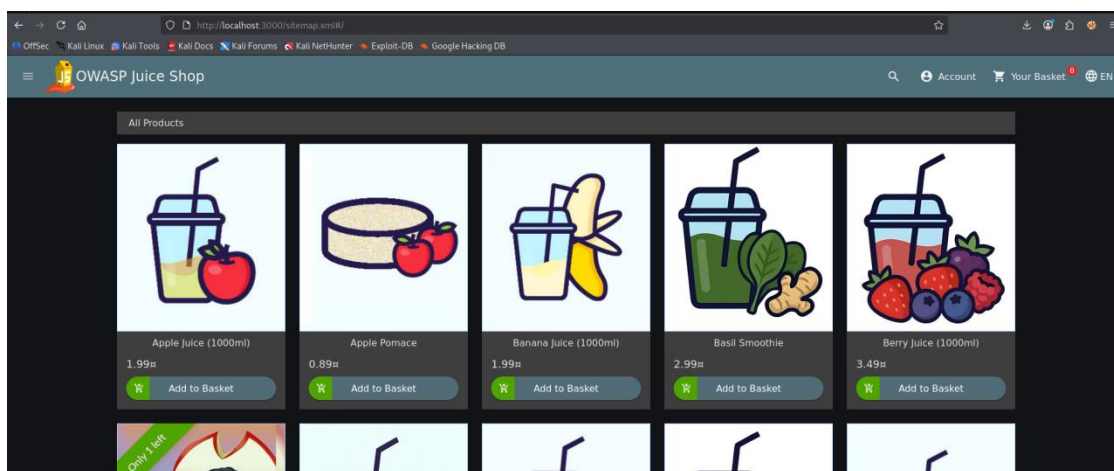


Figure 6 – `/sitemap.xml` intercepted by the Angular SPA router.

Activity 7: Browser DevTools – Network Traffic Analysis

Objective: Observe real network requests made by the application as it loads, to map backend API endpoints and static assets.

Why: In an SPA, most useful recon data (API paths, data flows) is only visible by watching what the app actually calls, not by crawling server-rendered HTML.

Input: `http://localhost:3000/#/polyfills.js` (app loaded, DevTools open)

Tool: Firefox/Chromium Developer Tools – Network tab

Command: Manual: F12 → Network tab → reload page

Output: Requests observed include GET `/rest/admin/application-configuration` (200), GET `/api/Quantitys/` (304), GET `/search?q=` (304), Socket.IO polling/websocket handshake, favicon, and numerous product image requests (`apple_juice.jpg`, `banana_juice.jpg`, `berry_juice.png`, etc.).

Observation: The app uses a REST API under `/rest/` and `/api/` prefixes, plus a Socket.IO channel for real-time features; several image requests were cancelled (NS_BINDING_ABORTED) as the page continued loading.

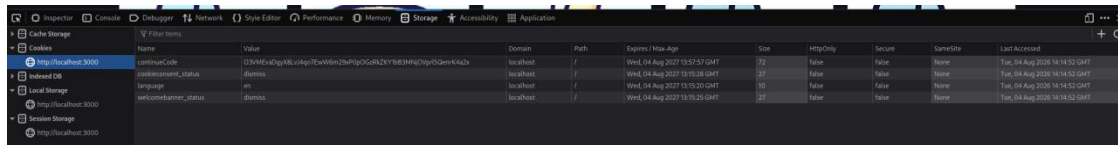
Why it matters: Builds a concrete list of backend endpoint prefixes and confirms real-time communication channels — core inputs for the attack-surface map.

Decision: Review the client-side JavaScript source directly to find additional endpoints not triggered during normal browsing.

Observation: No authentication/session token was present in cookies at this stage (pre-login); state such as the shopping basket is tracked client-side via Session Storage rather than server-side sessions.

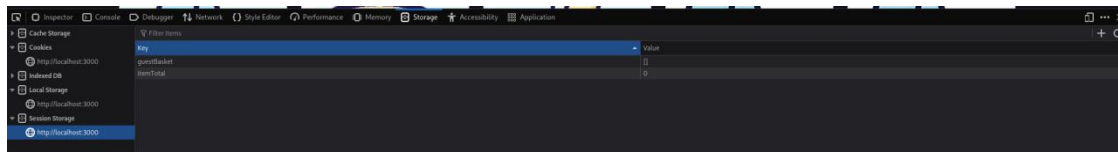
Why it matters: Establishes how the app manages guest vs authenticated state — relevant baseline for any later session-handling assessment.

Decision: Inspect a specific backend response in detail to look for further disclosure in response headers/body.



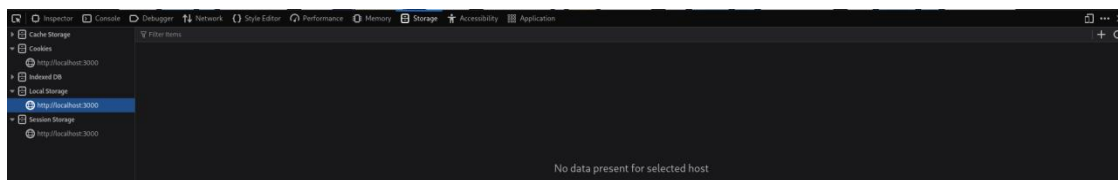
Filter Items	Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
Cache Storage										
Cookies										
http://localhost:3000	connectCode	C2b4b0a9ggr8Lk0qptCwW0n2b0p0G0A2K7b8JHq0p0G0emkA2k	localhost	/	Wed, 04 Aug 2027 15:57:52 GMT	72	False	False	None	Tue, 04 Aug 2026 14:14:52 GMT
Indexed DB										
Local Storage	authentication_status	anon	localhost	/	Wed, 04 Aug 2027 15:56:20 GMT	31	False	False	None	Tue, 04 Aug 2026 14:14:52 GMT
Session Storage	authentication_status	anon	localhost	/	Wed, 04 Aug 2027 15:56:20 GMT	27	False	False	None	Tue, 04 Aug 2026 14:14:52 GMT

Figure 9a – Cookies set for localhost:3000.



Filter Items	Key	Value
Cache Storage		
Cookies		
http://localhost:3000		
Indexed DB		
Local Storage		
Session Storage	guestBasket	{}
Session Storage	itemTotal	0

Figure 9b – Session Storage contents (guestBasket, itemTotal).



Filter Items	Key	Value
Cache Storage		
Cookies		
http://localhost:3000		
Indexed DB		
Local Storage		
Session Storage		

No data present for selected host.

Figure 9c – Local Storage empty at time of capture.

Activity 10: Response Header Deep Dive on a Discovered Endpoint

Objective: Examine the full response headers of a specific application-called endpoint observed during traffic analysis.

Why: Full header inspection (beyond curl -I) can reveal caching behaviour, content negotiation, and other headers not always visible at the top level.

Input: GET http://localhost:3000/rest/admin/application-configuration

Tool: Browser DevTools – Network tab

Command: Manual: inspect request in Network tab → Headers sub-tab

Output: Status 200 OK, HTTP/1.1, 9.52kB transferred (23.58kB decoded), Content-Type: application/json; charset=utf-8, Content-Encoding: br, Referrer-Policy: strict-origin-when-cross-origin, plus the same X-Content-Type-Options, X-Frame-Options, and X-Recruiting headers seen previously.

Observation: This endpoint is called automatically by the SPA on load (not manually probed) and returns JSON configuration data with a 200 status; response is Brotli-compressed.

Why it matters: Confirms the app exposes a configuration-related endpoint under /rest/admin/ that responds without any visible interactive authentication step during normal browsing — a point worth flagging for the next (enumeration) phase, but not investigated further here as exploitation is out of scope.

Decision: Run an automated crawler to systematically enumerate all reachable static assets and routes.

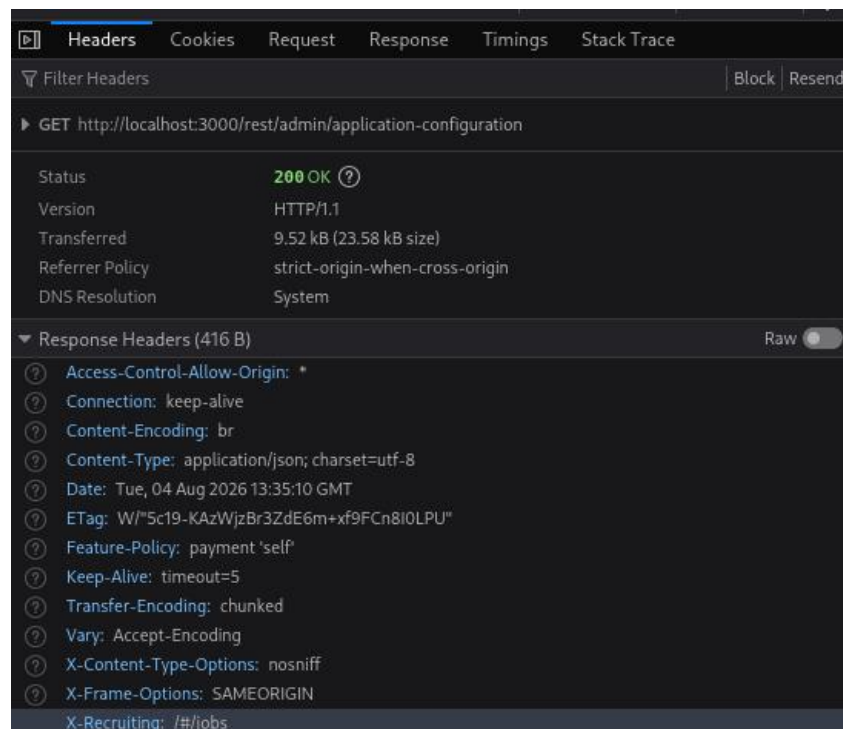


Figure 10 – Full response headers for /rest/admin/application-configuration.

Activity 11: Automated Crawling

Objective: Systematically enumerate reachable URLs, scripts, and static resources beyond what manual browsing surfaced.

Why: An automated crawler exhaustively follows links/asset references far faster and more completely than manual browsing, rounding out the attack-surface map.

Input: http://localhost:3000

Tool: Katana (ProjectDiscovery) v1.6.1

Command: `katana -u http://localhost:3000 -o katana_output.txt`

Output: Discovered: /scripts.js, multiple hashed chunk-*.js bundle files, /polyfills.js, /styles.css, /main.js, and an encoded path /%7B%7Bhref%7D%7D (a Handlebars-style template placeholder leaking into a link).

Observation: The encoded '{{href}}' path is a template artifact rather than a real page — indicates a link on the site was rendered with an unpopulated template variable; worth flagging as a minor front-end defect, not a security finding in itself.

Why it matters: Produces a comprehensive, de-duplicated inventory of static assets and confirms no additional hidden HTML routes were missed by manual review.

Decision: Recon objectives satisfied — consolidate all findings into the final report; hand off the endpoint/asset inventory as input to the (out-of-scope) enumeration phase.

```
(dharmik@kali)-[~]
$ katana -u http://localhost:3000 -o katana_output.txt

projectdiscovery.io

[INF] Current katana version v1.6.1 (latest)
[INF] Started standard crawling for => http://localhost:3000
http://localhost:3000
http://localhost:3000/scripts.js
http://localhost:3000/chunk-YVDT5JXT.js
http://localhost:3000/chunk-VS3A3LTT.js
http://localhost:3000/chunk-QBYXNN7Z.js
http://localhost:3000/chunk-OKA37M7B.js
http://localhost:3000/chunk-UNFVUBM2.js
http://localhost:3000/chunk-DYXK4NW4.js
http://localhost:3000/chunk-NWDAIMF4.js
http://localhost:3000/chunk-5K74DZ2F.js
http://localhost:3000/polyfills.js
http://localhost:3000/chunk-VJL3IV30.js
http://localhost:3000/chunk-PX7UKXVL.js
http://localhost:3000/%7B%7Bhref%7D%7D
http://localhost:3000/styles.css
http://localhost:3000/main.js
http://localhost:3000/main.js
```

Figure 11 – Katana crawl output listing discovered assets and paths.

3. Tools Used

Tool	Type	Purpose in This Engagement
Docker	Environment / Deployment	Deploy the OWASP Juice Shop container locally for an isolated test target.
Web Browser (Firefox)	Manual Testing	Visual verification, DevTools-based traffic, storage, and source inspection.
curl	CLI / HTTP client	Lightweight HTTP header inspection without rendering the page.
WhatWeb	CLI / Fingerprinting	Automated technology and framework fingerprinting.
Browser DevTools	Built-in browser tooling	Network traffic capture, JS source review, cookie/storage inspection.
Katana	CLI / Web Crawler	Automated crawling to enumerate static assets and routes.

4. Commands Executed

#	Command	Purpose
1	<code>docker pull bkimminich/juice-shop</code>	Download the target application image.
2	<code>docker images</code>	Confirm the image was downloaded successfully.
3	<code>curl -I http://localhost:3000</code>	Retrieve HTTP response headers only.
4	<code>whatweb http://localhost:3000</code>	Fingerprint web technologies in use.
5	<code>katana -u http://localhost:3000 -o katana_output.txt</code>	Crawl the application and save discovered URLs to file.

In addition to the commands above, several activities were performed manually through the browser and its Developer Tools (navigating to `/robots.txt` and `/sitemap.xml`, and using the Network, Debugger, and Storage panels), as these do not correspond to a single CLI command but are documented in full under Section 2.

5. Outputs

Activity	Output Artifact	Key Data Captured
Environment Setup	Docker CLI output	Image ID, size (338MB), successful layer pulls.
Reachability Check	Browser screenshot	Rendered storefront confirming live target.
HTTP Headers	curl output	Status 200, security headers, X-Recruiting header.
Fingerprinting	WhatWeb output	HTML5/SPA, uncommon headers, page title.
robots.txt	Text file content	Disallow: /ftp
sitemap.xml	Browser render	SPA router intercept — no real sitemap present.
Network Traffic	DevTools capture	/rest/ and /api/ endpoint calls, Socket.IO handshake.
Source Review	DevTools Sources snippets	/api/Products construction; password policy logic.
Cookies/Storage	DevTools Storage panel	4 cookies, Session Storage basket state, empty Local Storage.
Header Deep Dive	DevTools Headers panel	Full response header set for /rest/admin/application-configuration.
Crawl	katana_output.txt	List of JS bundles, CSS, and a malformed template path.

6. Observations

- The target is a Node.js/Express backend serving an Angular single-page application (confirmed independently via curl, WhatWeb, and DevTools).
- The application exposes at least two API path prefixes: /rest/ and /api/.
- A real-time channel via Socket.IO is established on every page load.
- robots.txt discloses a single disallowed path, /ftp, which is unusual for a modern SPA and does not correspond to any UI-linked feature.
- sitemap.xml is not a genuine XML sitemap — the client-side router intercepts the path and renders the normal storefront, which could mislead naive automated tooling into treating it as a valid page.
- The endpoint /rest/admin/application-configuration returns a 200 OK with JSON configuration data automatically on page load, without any visible interactive authentication step in this observation.
- Client-side JavaScript reveals password policy logic (minimum length 8, multi-tier strength ranges) directly in the bundled source.
- Session/guest state (e.g. shopping basket) is held in browser Session Storage rather than exclusively server-side; Local Storage was empty at capture time.
- All observed cookies are non-HttpOnly and non-Secure with SameSite=None, which is standard for this local HTTP (non-TLS) lab setup but would warrant attention in a hardened deployment.
- The automated crawl surfaced a malformed link containing an unpopulated template placeholder (%7B%7Bhref%7D%7D), indicating a front-end template rendering defect rather than a security issue.
- An unexpected scoreboard notification ('Error Handling' challenge) appeared once during testing — see Section 8 for details; this was not the result of deliberate exploitation.

7. Decision Process

Each step in this engagement was chosen based on the output of the previous one, following a broaden-then-narrow approach typical of manual reconnaissance:

- Started with environment setup and a visual check to establish that a valid, reachable target existed before running any tooling.
- Moved to the lowest-risk, lowest-effort check first (HTTP headers via curl) before layering on more specific tools.
- Used WhatWeb to independently corroborate the technology assessment made from headers alone, reducing the chance of a false read.
- Checked the two standard passive-disclosure files (robots.txt, sitemap.xml) next, since they require no tooling and sometimes reveal structure immediately.
- Because sitemap.xml revealed SPA routing behaviour rather than real data, the workflow pivoted from server-side file checks to client-side analysis (DevTools) to accurately map an SPA's real attack surface.
- Within DevTools, traffic (Network tab) was reviewed before source code (Sources/Debugger), since traffic shows what the app actually does, while source review fills in paths that aren't triggered during normal use.
- Cookie/storage inspection followed naturally from the traffic and source review, since both had already surfaced session-related behaviour worth confirming directly.
- A single endpoint (application-configuration) was chosen for a full header deep-dive because it was the most information-dense response observed and automatically triggered — a good representative sample rather than testing every endpoint manually.
- Automated crawling (Katana) was run last, as a systematic sweep to catch anything manual browsing missed, and to produce a machine-readable artifact list suitable for handoff to later phases.

8. Challenges Faced

- sitemap.xml did not behave as a traditional static file would; distinguishing 'SPA router intercept' from 'real content' required manually checking the rendered page against the URL bar rather than trusting the HTTP status alone.
- During normal interaction with the application, a scoreboard notification ('You successfully solved a challenge: Error Handling') appeared unexpectedly. This is a built-in Juice Shop gamification feature and was triggered incidentally, not through any deliberate exploitation attempt — it is flagged here for transparency and to note that even passive/normal use of a deliberately vulnerable app can trip its built-in challenge tracker.
- Several image requests in the Network tab showed as cancelled (NS_BINDING_ABORTED), which initially looked like errors but were determined to be the browser aborting off-screen image loads as the page continued rendering — a normal browser optimisation, not a target-side issue.
- Distinguishing genuinely interesting endpoints (e.g. /rest/admin/application-configuration) from routine application chatter required cross-referencing multiple observation points (Network tab, response headers, and source review) rather than relying on any single tool.

9. Lessons Learned

- For single-page applications, server-side file checks (robots.txt, sitemap.xml) only tell part of the story — browser DevTools-based traffic and source analysis is essential to see the real attack surface.
- Corroborating findings across multiple independent tools (e.g. curl and WhatWeb agreeing on headers) increases confidence and catches tool-specific blind spots.
- Reconnaissance against a gamified vulnerable-by-design app requires extra care to document any incidental side effects (like challenge triggers) transparently, since scope discipline matters even when the target is meant to be tested.
- A structured, repeatable documentation format (Objective/Why/Input/Tool/Command/Output/Observation/Decision) makes it straightforward to later translate manual steps into an automated script, since each activity's input and expected output are already explicit.

10. Recommendations for Automation

The following steps from this manual workflow are strong candidates for scripting into a repeatable recon automation tool:

- Wrap environment setup (docker pull / docker run / readiness check) in a single provisioning script that polls the target URL until it returns HTTP 200 before proceeding.
- Automate the curl header check and WhatWeb scan as sequential, unattended steps, writing results to timestamped output files for diffing across runs.
- Script robots.txt and sitemap.xml retrieval with response-content validation (e.g. detect HTML vs. XML content-type) to automatically flag SPA-router-intercepted 'fake' sitemaps like the one found here, rather than treating any 200 response as a valid file.
- Use a headless browser (e.g. Playwright/Puppeteer) to automate the DevTools-equivalent steps: capture all network requests during page load, extract unique endpoint paths (/rest/, /api/ prefixes), and dump cookies/localStorage/sessionStorage programmatically.
- Integrate static analysis of bundled JS (e.g. regex/AST scanning for endpoint string patterns and policy constants) to automate the client-side source review step.

- Schedule Katana (or an equivalent crawler) as the final automated step, with output automatically diffed against the previous run to highlight newly discovered assets or routes.
- Consolidate all tool outputs into a single structured format (e.g. JSON) so that the eventual report generation itself can be automated from raw tool output, keeping the human-readable report as a rendered view rather than the source of truth.